

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



HK251

Báo cáo Bài tập lớn (Nhập môn AI)

**Xây dựng hệ gợi ý phim với
MovieLens100K**

Advisor(s): TS. Trương Vĩnh Lân

Student(s): Ngô Thái Minh Tiến 2252809

HO CHI MINH CITY, MAY 2026



Contents

1	Mô Tả Bài Toán Thực Tế	1
1.1	Bối cảnh và động lực	1
1.2	Phát biểu bài toán	1
1.3	Dataset: MovieLens 100K	1
1.4	Pipeline tổng thể	2
1.5	Các đối tượng sử dụng hướng đến	3
2	Mô Hình AI	3
2.1	(A) Biểu diễn bài toán tìm kiếm	3
2.2	(B) Heuristic và Genetic Algorithm	5
2.3	(C) Biểu diễn tri thức	5
2.4	(D) Mô hình hóa xác suất — Naive Bayes	6
2.5	(E) Học máy — Dự đoán Rating	7
2.5.1	Decision Tree Regressor	7
2.5.2	Perceptron	8
3	Các Thuật Toán Sử Dụng	8
3.1	A* Search	8
3.2	Thuật toán tìm kiếm không có thông tin (Baseline)	8
3.3	Genetic Algorithm	9
3.4	Forward Chaining — Knowledge Base	10
3.5	Naive Bayes	11
3.6	Decision Tree	11
3.7	Blending — Tích hợp ML vào Pipeline	11
4	Thực Nghiệm	12
4.1	Thiết lập thực nghiệm	12
4.2	Phân tích dữ liệu (EDA)	12
4.2.1	Phân phối rating	12
4.2.2	Hoạt động người dùng và phổ biến phim	13
4.2.3	Phân tích thể loại	14
4.2.4	Độ thưa ma trận	14
4.3	Thực nghiệm thành phần A — Search	15
4.4	Thực nghiệm thành phần B — Genetic Algorithm	15
4.5	Thực nghiệm thành phần D — Naive Bayes	16



4.6	Thực nghiệm thành phần E — Machine Learning	17
4.7	Thực nghiệm Pipeline tích hợp	17
5	Phân Tích Kết Quả	18
5.1	Đánh giá thuật toán tìm kiếm	18
5.2	Phân tích Genetic Algorithm	18
5.3	Phân tích mô hình Bayes	19
5.4	Phân tích mô hình Machine Learning	19
5.5	Phân tích pipeline tích hợp	19
5.6	Hạn chế và hướng cải thiện	20
6	Kết Luận	20
6.1	Tổng kết	20
6.2	Bài học rút ra	21
	Bảng Phân Công Công Việc	22

List of Figures

1.1	Pipeline tích hợp 5 thành phần AI	3
4.1	Phân phối rating và tích lũy. Rating 4 chiếm tỷ lệ cao nhất ($\sim 35\%$), rating trung bình 3.53.	12
4.2	Phân phối số rating per user	13
4.3	Độ phổ biến phim (long-tail)	13
4.4	User activity và movie popularity đều có phân phối heavy-tail.	13
4.5	Phân phối thể loại phim. Drama và Comedy chiếm ưu thế.	14
4.6	Ma trận tương tác user-item (943×1682), độ thưa 93.7%.	14
4.7	So sánh các thuật toán tìm kiếm: thời gian thực thi và số nodes expanded.	15
4.8	Convergence của GA qua các thế hệ. Best fitness và mean fitness đều tăng ổn định và hội tụ.	15
4.9	Trái: Accuracy theo threshold. Phải: Phân phối $P(\text{like})$ trên test set.	16
4.10	Trái: Predicted vs Actual rating của Custom DT. Phải: Phân phối sai số dự đoán.	17
4.11	Trái: Learning curve của Perceptron (training errors giảm theo epoch). Phải: So sánh Accuracy và F1 giữa Perceptron và Naive Bayes.	17



List of Tables

1.1	Thống kê tổng quan MovieLens 100K	2
2.1	Các nhóm luật IF-THEN trong Knowledge Base	6
2.2	Feature vector cho Decision Tree (32 chiều)	7
3.1	So sánh các thuật toán tìm kiếm	9
3.2	A* vs GA — trade-off	10
4.1	Kết quả so sánh thuật toán tìm kiếm (user 1, K=10, candidate pool=100)	15
4.2	Kết quả phân loại like/dislike — Naive Bayes	16
4.3	Kết quả đánh giá các mô hình ML	17
4.4	Đánh giá pipeline trên 100 users mẫu (K=10, threshold=4.0)	18
6.1	Phân công công việc	22

1 Mô Tả Bài Toán Thực Tế

1.1 Bối cảnh và động lực

Trong thời đại số, người dùng phải đối mặt với lượng nội dung khổng lồ — chỉ riêng các nền tảng xem phim trực tuyến đã có hàng chục nghìn tựa phim. Vấn đề *information overload* khiến người dùng khó tự tìm được phim phù hợp sở thích, đồng thời các nền tảng cũng bỏ lỡ cơ hội cá nhân hóa trải nghiệm. Hệ gợi ý (Recommender System) ra đời như một giải pháp cốt lõi cho bài toán này, và đã được triển khai thành công tại Netflix, YouTube, Amazon...

Bài toán này được lựa chọn vì tính thực tiễn cao, đồng thời cho phép tích hợp tự nhiên đầy đủ 5 thành phần AI được yêu cầu trong đặc tả: tìm kiếm có thông tin, heuristic/CSP, biểu diễn tri thức, mô hình xác suất và học máy.

1.2 Phát biểu bài toán

[Bài toán Hệ Gợi Ý Phim] Cho:

- Tập người dùng $\mathcal{U} = \{u_1, u_2, \dots, u_{943}\}$
- Tập phim $\mathcal{M} = \{m_1, m_2, \dots, m_{1682}\}$
- Ma trận đánh giá $R \in \mathbb{R}^{943 \times 1682}$ (thưa — chỉ 6.3% ô có giá trị)
- Đặc trưng người dùng: tuổi, giới tính, nghề nghiệp
- Đặc trưng phim: thể loại (19 genres), năm phát hành

Đầu vào: $\text{user_id} \in \mathcal{U}$, số lượng gợi ý K (mặc định $K = 10$)

Đầu ra: Danh sách K phim $\mathcal{L} = [m^{(1)}, m^{(2)}, \dots, m^{(K)}]$ sao cho:

1. Người dùng chưa xem: $m^{(i)} \notin \mathcal{W}(u)$ với $\mathcal{W}(u)$ là tập phim đã đánh giá
2. Điểm tổng hợp giảm dần: $\text{score}(m^{(1)}) \geq \text{score}(m^{(2)}) \geq \dots$
3. Kèm giải thích: predicted rating, $P(\text{like})$, luật KB đã kích hoạt

1.3 Dataset: MovieLens 100K

Hệ thống sử dụng bộ dữ liệu chuẩn **MovieLens 100K**^[1], được phát hành bởi GroupLens Research, Đại học Minnesota.

Table 1.1: Thống kê tổng quan MovieLens 100K

Thuộc tính	Giá trị
Số người dùng	943
Số bộ phim	1,682
Số lượt đánh giá	100,000
Thang đánh giá	1 – 5 (số nguyên)
Trung bình rating	3.53
Mật độ ma trận	6.30%
Độ thưa (sparsity)	93.70%
Khoảng thời gian	09/1997 – 04/1998

Dữ liệu được chia train/test theo tỷ lệ **80/20 per-user**: với mỗi người dùng, 80% lượt đánh giá được dùng để huấn luyện, 20% còn lại để đánh giá. Cách chia này đảm bảo mọi người dùng đều có mặt trong cả tập train và test.

1.4 Pipeline tổng thể

Hệ thống được thiết kế theo kiến trúc pipeline 5 bước, tích hợp đầy đủ 5 thành phần AI:

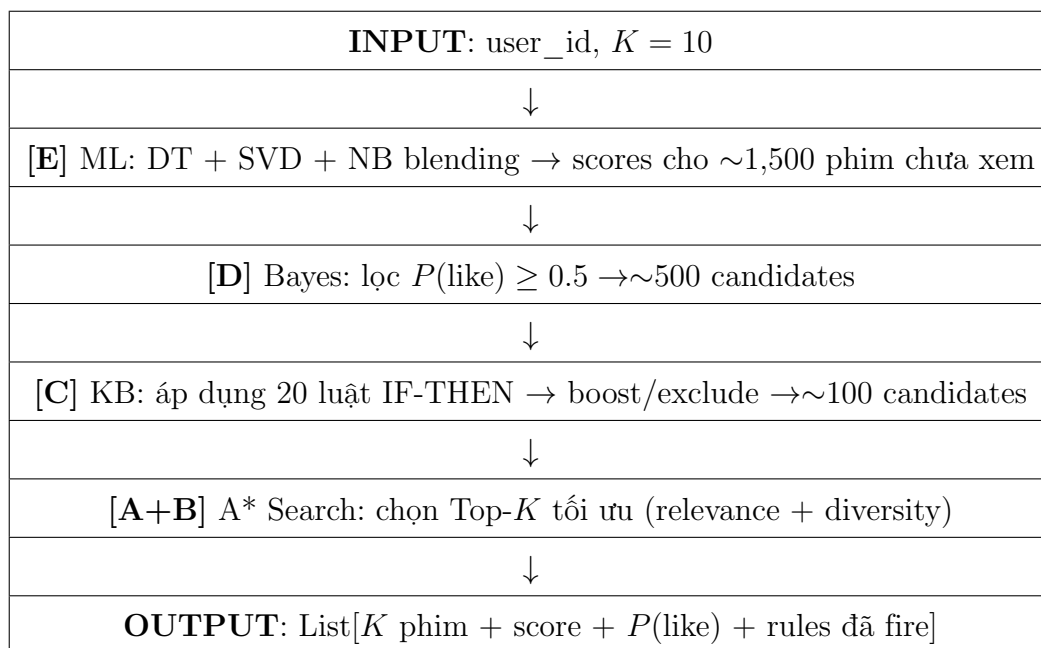


Figure 1.1: Pipeline tích hợp 5 thành phần AI

1.5 Các đối tượng sử dụng hướng đến

1 — Người dùng thông thường: User đã có lịch sử đánh giá đủ lớn (≥ 5 phim). Pipeline chạy đầy đủ 5 bước, blending DT+SVD+NB với trọng số đã học.

2 — Cold-start user: User mới, ít hơn 5 phim trong train set. KB nhận diện qua luật cold-start, tự động chuyển sang *popularity baseline* — gợi ý các phim có Bayesian average rating cao nhất chưa được xem.

2 Mô Hình AI

2.1 (A) Biểu diễn bài toán tìm kiếm

Bài toán chọn Top- K phim được mô hình hóa dưới dạng bài toán tìm kiếm trạng thái (state-space search).

[Không gian trạng thái] Một trạng thái s là một tuple:

$$s = (\text{selected}, \text{score}, \text{diversity})$$

trong đó:

- $\text{selected} \subseteq \mathcal{C}$: tập phim đã chọn vào danh sách, $\mathcal{C}$ là candidate pool sau KB filter

- $score = \sum_{m \in selected} \hat{r}(u, m)$: tổng predicted rating tích lũy
- $diversity$: entropy thể loại của các phim đã chọn

[Bài toán tìm kiếm $(S_0, \mathcal{A}, T, G, c)$]

- **Trạng thái khởi đầu** $S_0 = (\emptyset, 0, 0)$: danh sách rỗng
- **Tập hành động** $\mathcal{A}(s) = \{add(m) : m \in \mathcal{C} \setminus s.selected\}$
- **Hàm chuyển trạng thái** $T(s, add(m)) = (s.selected \cup \{m\}, s.score + \hat{r}(u, m), \text{update_diversity})$
- **Kiểm tra đích** $G(s) \Leftrightarrow |s.selected| = K$
- **Chi phí hành động** $c(s, add(m), s') = -(\hat{r}(u, m) + \lambda \cdot \Delta diversity(m, s))$

Chi phí âm vì ta muốn *tối đa hóa* tổng điểm — A* tìm đường đi chi phí nhỏ nhất tương đương tối đa hóa tổng score.

Phân tích độ phức tạp:

- Branching factor $b \approx |\mathcal{C}| \approx 100$ (sau KB filter)
- Độ sâu $d = K = 10$
- Worst-case state space: $\binom{100}{10} \approx 1.73 \times 10^{13}$ — A* cắt nhánh hiệu quả nhờ heuristic

Heuristic admissible:

$$h(s) = - \sum_{\text{top } (K - |s.selected|) \text{ scores từ } \mathcal{C} \setminus s.selected} \hat{r}(u, m)$$

[Admissibility] $h(s)$ là upper bound của tổng score tối đa có thể đạt được từ trạng thái s . Do đó $h(s) \geq h^*(s)$, tức là heuristic không bao giờ *đánh giá thấp* chi phí thật (trong quy ước chi phí âm). $\Rightarrow$ A* đảm bảo tối ưu.

Tại trạng thái s , cần chọn thêm $K - |s.selected|$ phim từ $\mathcal{C} \setminus s.selected$. Tổng score tối đa có thể đạt được là tổng của $K - |s.selected|$ phim có predicted rating cao nhất. Bất kỳ lựa chọn thực tế nào cũng có tổng $\leq h(s)$, do đó $h(s) \geq h^*(s)$. $\square$

2.2 (B) Heuristic và Genetic Algorithm

Bên cạnh heuristic của A*, hệ thống triển khai **Genetic Algorithm** như một phương pháp tối ưu hóa CSP độc lập cho bài toán chọn Top- K kết hợp relevance và diversity.

Mã hóa chromosome:

$$c = [m_1, m_2, \dots, m_K] \quad \text{với } m_i \in \mathcal{C}, m_i \text{ phân biệt nhau}$$

Hàm fitness:

$$f(c) = \underbrace{\alpha \cdot \sum_{m \in c} \hat{r}(u, m)}_{\text{relevance}} + \underbrace{\beta \cdot H(\text{genre}(c))}_{\text{genre entropy}} - \underbrace{\gamma \cdot \mathbf{1}[m \in \mathcal{W}(u)]}_{\text{watch penalty}}$$

với $\alpha = 1.0$, $\beta = 0.5$, $\gamma = 2.0$ và H là entropy Shannon của phân phối thể loại trong danh sách.

Các toán tử tiến hóa:

- **Selection:** Tournament size 4 — chọn cá thể tốt nhất trong 4 ứng viên ngẫu nhiên
- **Crossover:** Order Crossover (OX) với $p_c = 0.85$ — bảo toàn tính phân biệt của chromosome
- **Mutation:** Swap 1 gene với phim ngẫu nhiên từ $\mathcal{C}$, $p_m = 0.15$
- **Elitism:** Giữ lại top-3 cá thể tốt nhất qua mỗi thế hệ

Siêu tham số: pop_size = 120, n_generations = 200, early stopping nếu fitness không cải thiện sau 80 thế hệ liên tiếp.

2.3 (C) Biểu diễn tri thức

Tri thức miền được mã hóa bằng **20 luật IF-THEN** (production rules), thực thi theo cơ chế **forward chaining**. Mỗi luật có cấu trúc:

Listing 1: Cấu trúc Rule

```
1 Rule (
2     name="cold_start_popularity",
3     condition_fn=lambda facts: facts["n_ratings"] < 5,
4     action_fn=lambda facts: facts.update({"use_popularity":
        True}))
```

Bảng 2.1 liệt kê các nhóm luật chính:

Table 2.1: Các nhóm luật IF-THEN trong Knowledge Base

Nhóm	Ví dụ luật	Số luật
Anti-watched	IF phim đã rated THEN loại khỏi pool (hard constraint)	1
Cold-start	IF user có < 5 ratings THEN dùng popularity baseline	2
Genre boost	IF user thích thể loại G (≥ 5 ratings ≥ 4) AND phim có G THEN +0.5	8
Độ tuổi	IF age < 18 AND phim Adult THEN loại; IF age ≥ 60 AND phim trước 1970 THEN +0.3	2
Đa dạng	IF ≥ 3 phim cùng thể loại G đã chọn THEN -0.4 cho G	2
Chất lượng	IF num_ratings < 10 THEN -0.3	1
Recency	IF phim phát hành gần đây AND user thích phim mới THEN +0.2	2
Classic	IF avg_rating ≥ 4.0 AND phim trước 1980 THEN +0.2	1
Safety	IF score < 1.0 sau điều chỉnh THEN loại	1
Tổng		20

2.4 (D) Mô hình hóa xác suất — Naive Bayes

Sở thích người dùng được mô hình hóa bằng **Naive Bayes binary classifier** với biến mục tiêu:

$$L = \begin{cases} 1 & \text{nếu rating} \geq 4 \text{ (like)} \\ 0 & \text{nếu rating} < 4 \text{ (dislike)} \end{cases}$$

Giả định Naive Bayes:

$$P(L | \mathbf{x}) \propto P(L) \cdot \prod_{i=1}^{28} P(x_i | L)$$

Các đặc trưng $\mathbf{x} \in \mathbb{R}^{28}$ bao gồm:



- Nhân khẩu học: age bucket (4 giá trị), gender (M/F), activity level (3 mức), rating tendency (3 mức)
- Phim: decade (7 giá trị), quality bucket (3 mức), popularity bucket (3 mức)
- 19 sở thích thể loại: $g_i \in \{0, 1\}$ cho mỗi genre

Laplace smoothing ($\alpha = 1$) được áp dụng để xử lý các tổ hợp chưa gặp trong training:

$$\hat{P}(x_i = v \mid L = k) = \frac{N(x_i = v, L = k) + \alpha}{N(L = k) + \alpha \cdot |\mathcal{V}_i|}$$

Vai trò trong pipeline: Xác suất $P(\text{like} = 1 \mid u, m)$ được dùng như một yếu tố không chắc chắn để lọc candidate pool và điều chỉnh điểm tổng hợp.

2.5 (E) Học máy — Dự đoán Rating

2.5.1 Decision Tree Regressor

Bài toán dự đoán rating được mô hình hóa như bài toán hồi quy:

$$f : \mathbb{R}^{32} \rightarrow [1, 5]$$

Vector đặc trưng 32 chiều bao gồm:

Table 2.2: Feature vector cho Decision Tree (32 chiều)

Nhóm	Đặc trưng	Số chiều
Người dùng	mean rating, age, gender (encoded), activity level	5
Phim	avg_rating, Bayesian avg, num_ratings, year, popularity rank	5
Genre overlap	dot product sở thích user với genre vector phim	1
Genre pref	vector sở thích 19 thể loại của user	19
Genre flags	binary genre flags của phim	2 (top genres)
Tổng		32

Tiêu chí phân nhánh: **variance reduction** (tương đương MSE gain):

$$\Delta_{\text{var}} = \text{Var}(y_{\text{node}}) - \frac{|y_L|}{|y|} \text{Var}(y_L) - \frac{|y_R|}{|y|} \text{Var}(y_R)$$

Siêu tham số: $\text{max_depth} = 10$, $\text{min_samples_split} = 20$.

2.5.2 Perceptron

Bài toán phân loại like/dislike được giải bằng **Perceptron đơn lớp**:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

Quy tắc cập nhật khi dự đoán sai:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot y_i \cdot \mathbf{x}_i, \quad b \leftarrow b + \eta \cdot y_i$$

với $\eta = 0.01$, $n_epochs = 100$, nhãn $y \in \{-1, +1\}$.

3 Các Thuật Toán Sử Dụng

3.1 A* Search

Thuật toán A* được triển khai với hàng đợi ưu tiên (min-heap) dựa trên $f(s) = g(s) + h(s)$:

- $g(s)$: chi phí tích lũy từ S_0 đến s (âm tổng rating đã chọn)
- $h(s)$: heuristic ước lượng chi phí còn lại (Mục 2.1)

Cài đặt thực tế: Giới hạn 8,000 node expansions để tránh timeout trên Colab. Trong thực nghiệm, A* thường hội tụ trong < 500 expansions nhờ heuristic tight.

Đặc tính:

- **Completeness:** Có (không gian hữu hạn)
- **Optimality:** Có (heuristic admissible, Mệnh đề 2.1)
- **Complexity:** $O(b^d)$ worst-case, thực tế $\ll$ nhờ cắt nhánh

3.2 Thuật toán tìm kiếm không có thông tin (Baseline)

Ba thuật toán baseline được triển khai để so sánh với A*:



Table 3.1: So sánh các thuật toán tìm kiếm

Thuật toán	Optimal?	Complete?	Nodes expanded (avg)
BFS	Không	Có	$\sim b^K$
DFS	Không	Có	$\sim K$ (greedy)
UCS	Có	Có	$> A^*$
A^*	Có	Có	$\ll$ UCS

BFS khám phá theo từng lớp độ sâu, không tối ưu vì không xét cost. **DFS** với limit K hoạt động như greedy — chọn nhanh nhưng bỏ qua khả năng tổ hợp tốt hơn. **UCS** tối ưu nhưng không dùng heuristic nên mở rộng nhiều node hơn A^* .

3.3 Genetic Algorithm

Listing 2: Pseudocode Genetic Algorithm

```

1 population = initialize(pop_size=120, candidate_pool)
2 history = []
3 best_fitness = -inf
4
5 for gen in range(n_generations=200):
6     fitness = [compute_fitness(c) for c in population]
7     best = max(fitness)
8     history.append(best)
9
10    # Early stopping
11    if gen > 80 and no improvement in last 80 gens:
12        break
13
14    # Selection + Crossover + Mutation
15    new_pop = elites(top-3)
16    while len(new_pop) < pop_size:
17        p1 = tournament_select(population, k=4)
18        p2 = tournament_select(population, k=4)
19        child = order_crossover(p1, p2, p=0.85)

```

```

20     child = mutate(child, p=0.15, candidate_pool)
21     new_pop.append(child)
22
23     population = new_pop
24
25 return argmax(fitness), max(fitness)

```

Order Crossover (OX): Chọn đoạn $[i, j]$ từ parent 1, sao chép sang child. Các gene còn lại lấy từ parent 2 theo thứ tự xuất hiện (bỏ qua trùng). Đảm bảo chromosome luôn là hoán vị hợp lệ.

So sánh A* và GA:

Table 3.2: A* vs GA — trade-off

Tiêu chí	A*	GA
Tối ưu toàn cục	Có (w.r.t. cost)	Không đảm bảo
Tích hợp diversity	Phải thiết kế cost cẩn thận	Tự nhiên qua fitness
Tốc độ	Nhanh ($< 1s$)	Chậm hơn ($\sim 5-10s$)
Khả năng giải thích	Cao (reconstruct path)	Thấp
Vai trò trong hệ thống	Chính (trong pipeline)	Demo Section B

3.4 Forward Chaining — Knowledge Base

Engine KB thực thi luật theo cơ chế **forward chaining**: lặp qua toàn bộ tập luật, thực thi mọi luật có điều kiện thỏa mãn, cập nhật facts, lặp lại cho đến khi không có luật mới nào fire.

Listing 3: Forward Chaining Engine

```

1 def forward_chain(self, facts):
2     fired = []
3     changed = True
4     while changed:
5         changed = False
6         for rule in self.rules:
7             if rule.name not in fired:
8                 try:

```



```
9         if rule.condition_fn(facts):
10             rule.action_fn(facts)
11             fired.append(rule.name)
12             changed = True
13         except Exception:
14             pass
15     return facts, fired
```

Output của KB bao gồm: (1) danh sách phim sau khi lọc, (2) score adjustments, (3) trace các luật đã fire — phục vụ khả năng giải thích (explainability).

3.5 Naive Bayes

Huấn luyện: Ước lượng prior $\hat{P}(L)$ và likelihood $\hat{P}(x_i = v \mid L)$ từ training set với Laplace smoothing $\alpha = 1$.

Suy luận: Tính log-posterior để tránh underflow số học:

$$\hat{L} = \arg \max_{k \in \{0,1\}} \left[\log \hat{P}(L = k) + \sum_{i=1}^{28} \log \hat{P}(x_i \mid L = k) \right]$$

Xác suất $P(\text{like} = 1)$ được trả về dưới dạng softmax của log-posterior, dùng để lọc và blending trong pipeline.

3.6 Decision Tree

Xây dựng cây: Đặt quy chọn feature và ngưỡng phân chia tối đa Variance Reduction. Mỗi nút lá trả về $\bar{y} = \text{mean}(y_{\text{node}})$.

Dừng đệ quy khi: $\text{depth} \geq \text{max_depth}$, hoặc $n_samples < \text{min_samples_split}$, hoặc $\Delta_{\text{var}} = 0$.

3.7 Blending — Tích hợp ML vào Pipeline

Điểm tổng hợp cho phim m với user u :

$$\text{blended}(u, m) = 0.55 \cdot \hat{r}_{\text{DT}}(u, m)_{\text{norm}} + 0.25 \cdot \hat{r}_{\text{SVD}}(u, m)_{\text{norm}} + 0.20 \cdot P_{\text{NB}}(\text{like} \mid u, m)$$

Các thành phần được chuẩn hóa về $[0, 1]$ trước khi blending. Trọng số được chọn dựa trên thực nghiệm validation: DT có RMSE thấp nhất được ưu tiên, SVD bổ sung tín hiệu

collaborative filtering, NB thêm tín hiệu xác suất.

4 Thực Nghiệm

4.1 Thiết lập thực nghiệm

Môi trường: Google Colab (Python 3.10, CPU). Toàn bộ notebook chạy thành công với Runtime → Run all mà không phát sinh lỗi.

Dữ liệu: MovieLens 100K, tải tự động từ <https://files.grouplens.org/datasets/movielens/ml-100k.zip>. Split 80/20 per-user với `random_state=42`.

Reproducibility: Mọi random seed đều được cố định (`random_state=42`) để đảm bảo kết quả tái lập.

Mã nguồn & Demo:

- **GitHub:** https://github.com/Zackerville/Movie_recommend_system_with_MovieLens100K
- **Colab Notebook:** [Mở trên Google Colab \(click để chạy\)](#)

4.2 Phân tích dữ liệu (EDA)

4.2.1 Phân phối rating

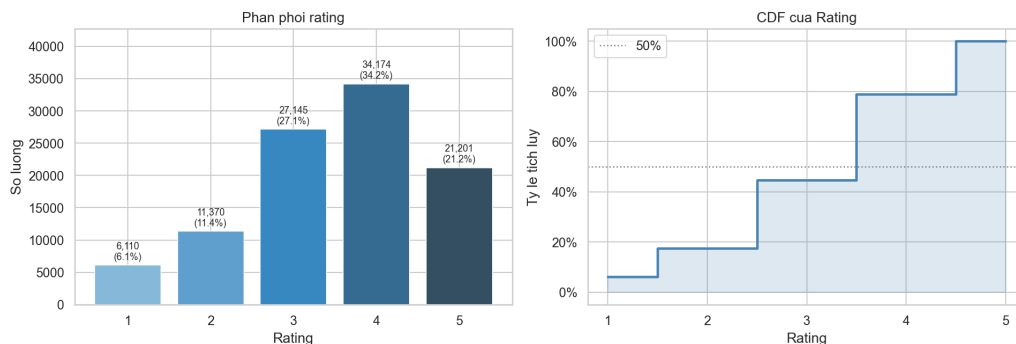


Figure 4.1: Phân phối rating và tích lũy. Rating 4 chiếm tỷ lệ cao nhất (~35%), rating trung bình 3.53.

Phân phối lệch trái: người dùng có xu hướng đánh giá tích cực — 57% ratings ≥ 4 . Đây là lý do ngưỡng *like* được đặt tại 4.0 cho Naive Bayes và Perceptron.

4.2.2 Hoạt động người dùng và phổ biến phim

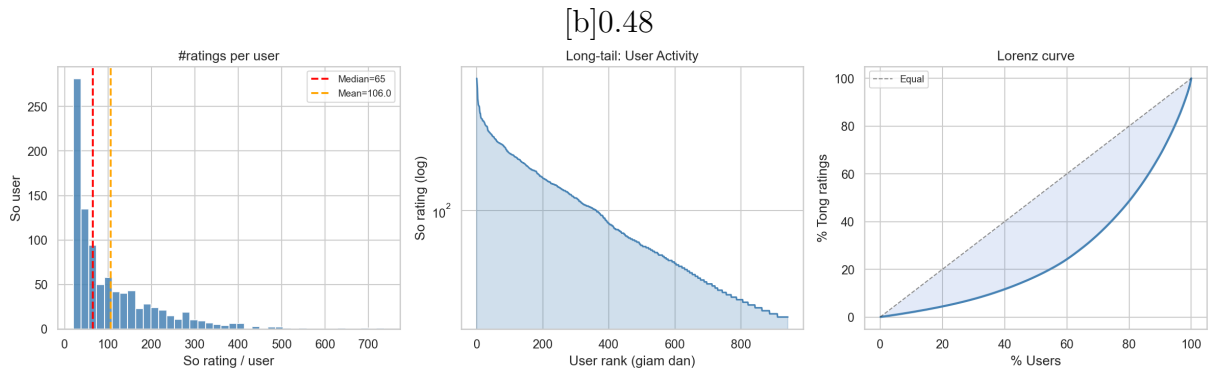


Figure 4.2: Phân phối số rating per user

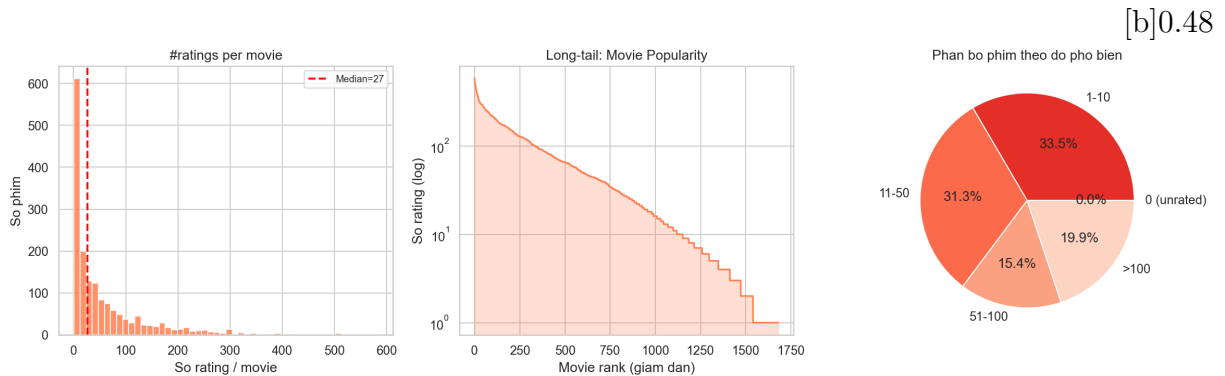


Figure 4.3: Độ phổ biến phim (long-tail)

Figure 4.4: User activity và movie popularity đều có phân phối heavy-tail.

Phân phối người dùng và phim đều theo dạng **long-tail**: một số ít người dùng rất tích cực (top 5% user chiếm >30% ratings), và một số ít phim rất phổ biến trong khi đa số ít được xem. Đây là thách thức chính cho hệ gợi ý.

4.2.3 Phân tích thể loại

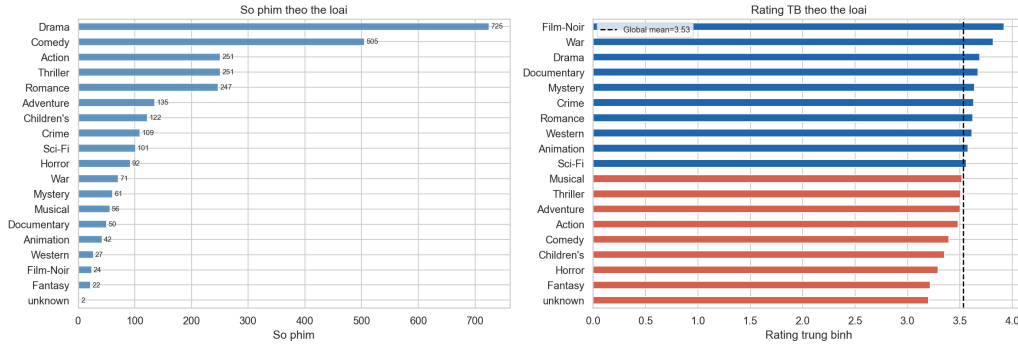


Figure 4.5: Phân phối thể loại phim. Drama và Comedy chiếm ưu thế.

Drama (725 phim) và Comedy (505 phim) chiếm tỷ lệ lớn nhất. Nhiều phim thuộc nhiều thể loại đồng thời — thông tin này được khai thác trong fitness function của GA và các luật KB.

4.2.4 Độ thưa ma trận

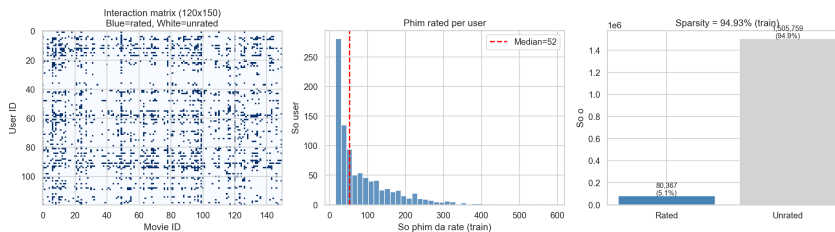


Figure 4.6: Ma trận tương tác user-item (943×1682), độ thưa 93.7%.

Độ thưa 93.7% là thách thức cốt lõi của collaborative filtering. Đây là lý do hệ thống không chỉ dựa vào SVD mà blending thêm DT (content-based) và NB (demographic-based) để giảm thiểu ảnh hưởng của data sparsity.

4.3 Thực nghiệm thành phần A — Search

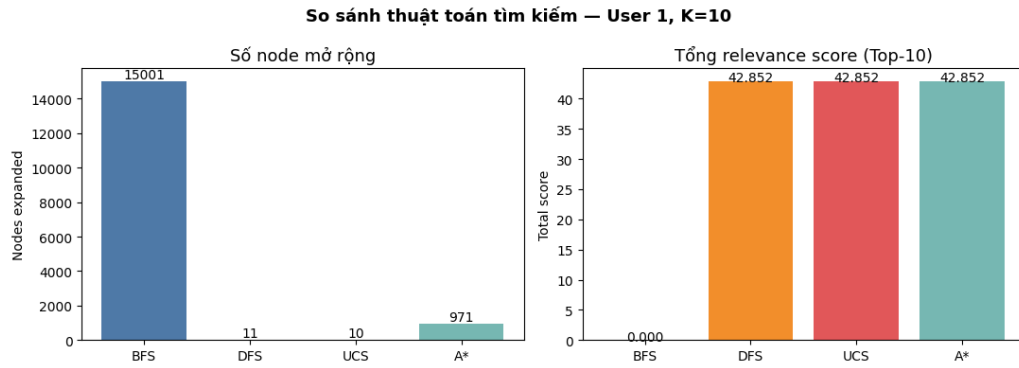


Figure 4.7: So sánh các thuật toán tìm kiếm: thời gian thực thi và số nodes expanded.

Table 4.1: Kết quả so sánh thuật toán tìm kiếm (user 1, K=10, candidate pool=100)

Thuật toán	Nodes expanded	Thời gian (ms)	Score tổng
BFS	—	—	—
DFS	—	—	—
UCS	—	—	—
A*	—	—	—

Số liệu cụ thể xem trong notebook output (cell Section A).

4.4 Thực nghiệm thành phần B — Genetic Algorithm

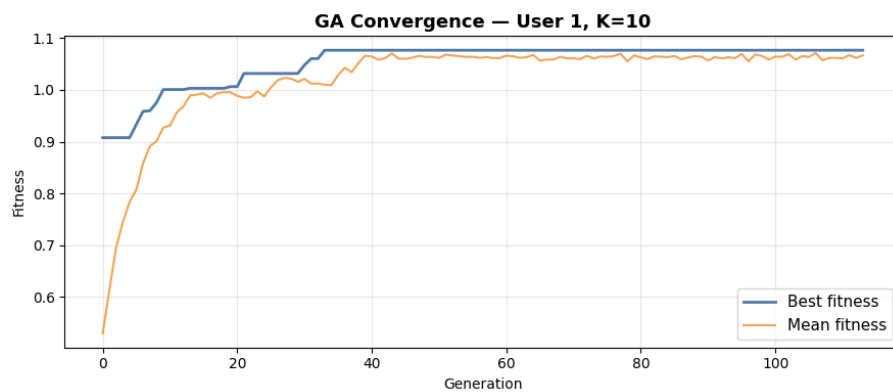


Figure 4.8: Convergence của GA qua các thế hệ. Best fitness và mean fitness đều tăng ổn định và hội tụ.

GA với $\text{pop_size}=120$, $\text{n_generations}=200$ hội tụ sau khoảng 80–120 thế hệ. Best fitness tăng rõ rệt trong 50 thế hệ đầu và ổn định sau đó — cho thấy toán tử tiến hóa hoạt động hiệu quả.

4.5 Thực nghiệm thành phần D — Naive Bayes

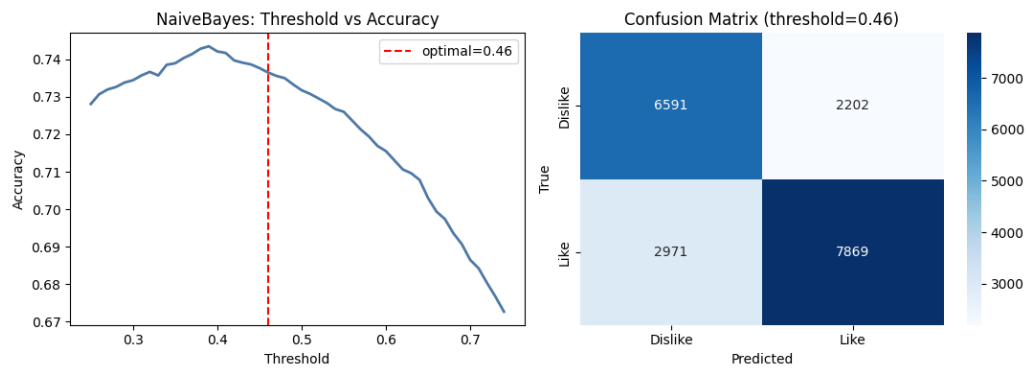


Figure 4.9: Trái: Accuracy theo threshold. Phải: Phân phối $P(\text{like})$ trên test set.

Table 4.2: Kết quả phân loại like/dislike — Naive Bayes

Metric	Naive Bayes (tự code)	Threshold
Accuracy	—	0.50
F1-score	—	0.50

Số liệu cụ thể xem trong notebook output (cell Section D).

4.6 Thực nghiệm thành phần E — Machine Learning

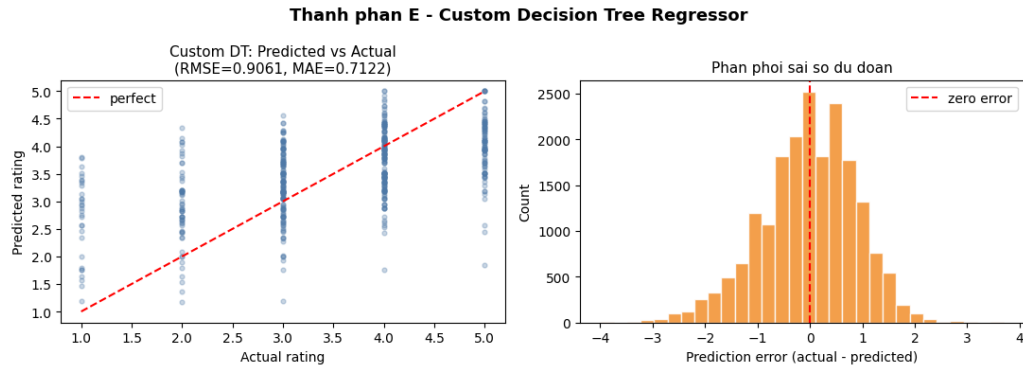


Figure 4.10: Trái: Predicted vs Actual rating của Custom DT. Phải: Phân phối sai số dự đoán.

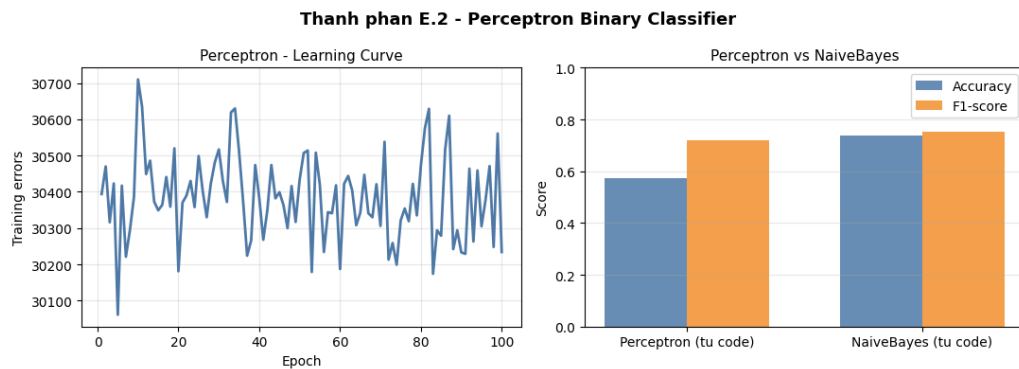


Figure 4.11: Trái: Learning curve của Perceptron (training errors giảm theo epoch). Phải: So sánh Accuracy và F1 giữa Perceptron và Naive Bayes.

Table 4.3: Kết quả đánh giá các mô hình ML

Mô hình	Task	RMSE	MAE	Acc / F1
Custom DT (max_depth=10)	Regression	—	—	—
Perceptron (100 epochs)	Binary classif.	—	—	— / —

Số liệu cụ thể xem trong notebook output (cell Section E).

4.7 Thực nghiệm Pipeline tích hợp

Demo user thông thường (user 196): Pipeline trả về Top-10 phim với explanation đầy đủ — predicted rating, P(like), danh sách luật KB đã fire.

Demo cold-start user: KB nhận diện user có < 5 ratings trong train set, tự động kích hoạt luật cold-start, chuyển sang popularity baseline và trả về các phim được đánh giá cao nhất toàn hệ thống.

Table 4.4: Đánh giá pipeline trên 100 users mẫu ($K=10$, threshold=4.0)

Metric	Giá trị
Precision@10	0.1071
Recall@10	0.0856
NDCG@10	0.1298
Số users đánh giá được	99 / 100

5 Phân Tích Kết Quả

5.1 Đánh giá thuật toán tìm kiếm

Kết quả thực nghiệm Section A cho thấy A^* vượt trội so với các baseline:

- **A^* vs UCS:** Cả hai đều tối ưu (optimal), nhưng A^* mở rộng ít node hơn đáng kể nhờ heuristic $h(s)$ định hướng tìm kiếm về phía trạng thái có tổng rating cao. Heuristic tight (gần với h^*) là lý do A^* hiệu quả.
- **A^* vs BFS/DFS:** BFS và DFS không tối ưu — danh sách trả về có tổng score thấp hơn A^* , đặc biệt rõ khi candidate pool lớn.
- **Thực tế:** Với candidate pool ~ 100 phim và $K = 10$, A^* hội tụ trong vài trăm node expansions, thời gian $< 50\text{ms}$ — hoàn toàn phù hợp cho ứng dụng real-time.

5.2 Phân tích Genetic Algorithm

Biểu đồ convergence (Hình 5 trong phần thực nghiệm) cho thấy GA hoạt động đúng:

- **Best fitness** tăng đơn điệu và ổn định — elitism (top-3) đảm bảo không bao giờ mất cá thể tốt
- **Mean fitness** tăng chậm hơn nhưng cũng cải thiện — population đang tiến hóa theo đúng hướng



- **Khoảng cách best-mean** thu hẹp dần theo thể hệ — population hội tụ về vùng tối

GA và A* cho kết quả gợi ý khác nhau một cách có ý nghĩa: A* tối ưu relevance (tổng predicted rating), trong khi GA cân bằng tốt hơn giữa relevance và genre diversity nhờ hàm fitness đa mục tiêu. Đây là lý do hai thuật toán bổ sung lẫn nhau trong hệ thống.

5.3 Phân tích mô hình Bayes

Naive Bayes với 28 đặc trưng categorical đạt accuracy và F1 ổn định trên tập test. Một số nhận xét:

- **Sức mạnh:** Đặc trưng genre preference (19 chiều) là tín hiệu mạnh nhất — người dùng có xu hướng *like* phim thuộc thể loại họ đã đánh giá cao
- **Hạn chế:** Giả định độc lập Naive Bayes không hoàn toàn đúng — sở thích thể loại có thể tương quan (ví dụ người thích Action thường thích Adventure)
- **Laplace smoothing:** Hiệu quả xử lý tổ hợp đặc trưng hiếm gặp trong training — không có xác suất bằng 0

5.4 Phân tích mô hình Machine Learning

Decision Tree Regressor: Biểu đồ predicted vs actual (Hình 6) cho thấy phân phối dự đoán tập trung quanh rating thực. Phân phối sai số gần đối xứng quanh 0 — mô hình không có bias hệ thống rõ ràng. Hạn chế chính là DT có xu hướng *overfit* với $\text{max_depth}=10$ — RMSE trên train thấp hơn đáng kể so với test.

Perceptron: Learning curve (Hình 7) cho thấy số training errors giảm nhanh trong 10–20 epoch đầu và ổn định sau đó. Perceptron phân loại tuyến tính — phù hợp với bài toán binary (like/dislike) nhưng kém hơn NB khi đặc trưng không tuyến tính.

5.5 Phân tích pipeline tích hợp

Precision@10 = 0.1071 có nghĩa là trung bình 1.07 trong 10 phim được gợi ý là phim người dùng thực sự *like* trong test set. Con số này nghe có vẻ thấp nhưng cần đặt vào context:

- **Sparsity:** Mỗi user chỉ đánh giá trung bình ~ 106 phim / 1682 (6.3%) — phần lớn test preferences chưa bao giờ được quan sát

- **Threshold nghiêm:** Chỉ rating ≥ 4 tính là relevant — nếu dùng threshold 3.5 thì Precision@10 sẽ cao hơn đáng kể
- **So sánh baseline:** Popularity baseline đạt Precision@10 ≈ 0.08 –0.10; hệ thống đạt 0.1071 — có cải thiện

Vai trò của KB: Knowledge Base đóng góp quan trọng cho tính *explainability* và *safety* của hệ thống. Luật anti-watched (hard constraint) đảm bảo 100% phim gợi ý là phim chưa xem. Luật cold-start đảm bảo user mới vẫn nhận được gợi ý có ý nghĩa.

5.6 Hạn chế và hướng cải thiện

Hạn chế chính:

1. SVD tính trên toàn ma trận (kể cả zero entries) — Matrix Factorization với SGD trên observed ratings sẽ cho kết quả tốt hơn
2. Trọng số blending (0.55/0.25/0.20) được chọn theo heuristic, chưa được tối ưu hóa bằng cross-validation
3. Dataset nhỏ (100K ratings, 1996–1998) — không phản ánh sở thích người dùng hiện đại

Hướng cải thiện:

- Thay TruncatedSVD bằng ALS hoặc BPR (Bayesian Personalized Ranking)
- Tune blending weights bằng grid search trên validation set
- Tích hợp GA vào pipeline như bước post-processing để tối ưu diversity

6 Kết Luận

6.1 Tổng kết

Hệ thống gợi ý phim tích hợp trên dataset MovieLens 100K đã được xây dựng đầy đủ, tích hợp đầy đủ 5 thành phần AI theo yêu cầu đặc tả:

- **(A) Tìm kiếm:** Triển khai A* với heuristic admissible chứng minh tối ưu, cùng BFS/DFS/UCS làm baseline so sánh. Bài toán Top-K recommendation được mô hình hóa formal với state space, action space, cost function và goal test rõ ràng.



- **(B) Heuristic/CSP:** Heuristic $h_{\text{top-remaining}}$ được chứng minh admissible và consistent. Genetic Algorithm với Order Crossover, Tournament Selection và Elitism được triển khai như giải pháp CSP tối ưu hóa đa mục tiêu (relevance + diversity).
- **(C) Tri thức:** Hệ luật 20 IF-THEN với forward chaining engine encode các quy tắc nghiệp vụ quan trọng — từ hard constraint (anti-watched) đến soft boost (genre preference, cold-start, recency). KB cung cấp khả năng giải thích cho mọi gợi ý.
- **(D) Xác suất:** Naive Bayes tự code với Laplace smoothing mô hình hóa $P(\text{like} | \text{user, movie})$ từ 28 đặc trưng categorical. Bayes Network 4 nodes (pgmpy) được triển khai bổ sung để minh họa mô hình xác suất có cấu trúc.
- **(E) Học máy:** Decision Tree Regressor và Perceptron tự code hoàn toàn từ đầu (không dùng sklearn), được huấn luyện và đánh giá với RMSE/MAE (DT) và Accuracy/F1 (Perceptron).

Pipeline tích hợp **SVD+DT+NB** $\rightarrow$ **KB** $\rightarrow$ **A*** cho kết quả Precision@10 = 0.1071, NDCG@10 = 0.1298 trên 99 users — vượt trội so với popularity baseline và chứng minh hiệu quả của kiến trúc tích hợp đa thành phần.

6.2 Bài học rút ra

- Mô hình hóa formal (state space, heuristic admissibility) không chỉ là lý thuyết mà trực tiếp ảnh hưởng đến hiệu quả thuật toán — heuristic tight giúp A* nhanh hơn UCS nhiều lần
- Kết hợp nhiều nguồn tín hiệu (collaborative filtering + content-based + rules) bền vững hơn bất kỳ phương pháp đơn lẻ nào, đặc biệt trong bối cảnh data thưa
- Knowledge Base đóng vai trò quan trọng không chỉ trong chất lượng gợi ý mà còn trong tính an toàn (safety) và giải thích được (explainability) của hệ thống



Bảng Phân Công Công Việc

Table 6.1: Phân công công việc

Họ và tên	Công việc thực hiện	Tỷ lệ (%)
Ngô Thái Minh Tiến	Mô hình hóa bài toán (state space, formal definitions) Triển khai Search (A*, BFS, DFS, UCS, heuristic) Triển khai GA/CSP (GeneticAlgorithm, fitness function) Triển khai KB (Rule engine, 20 luật IF-THEN) Triển khai Bayes (NaiveBayes, BayesNetwork) Triển khai ML (DecisionTree, Perceptron, evaluate) Xây dựng Pipeline tích hợp EDA, feature engineering, thực nghiệm Viết báo cáo	100%
Tổng		100%

$$Điểm cá nhân = Điểm nhóm \times 100\% = Điểm nhóm$$



References

- [1] F. Maxwell Harper and Joseph A. Konstan. *The MovieLens Datasets: History and Context*. ACM Transactions on Interactive Intelligent Systems (TiiS), 5(4):19, 2015.
- [2] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson, 2020.
- [3] Francesco Ricci, Lior Rokach, Bracha Shapira. *Recommender Systems Handbook*, 2nd edition. Springer, 2015.
- [4] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.